

Misattribution Machine

Technical Implementation

Code Reference & System Architecture

May 2026

STEP 1 — DATA PROCESSING PIPELINE

Two Excel datasets were converted to JSON using openpyxl. The processing script reads each sheet, normalises column headers, strips whitespace, infers the artist from the filename, and outputs structured JSON.

Output files:

- artworks_data.json — 354 records (Malevich, Exter, Pagava, Kakabadze)
- parajanov_data.json — 305 bibliographic records

Each record contains: ID, Title, Artist, Holding Institution, Flawed nationality label, Correct nationality label, Correct name form, Notes on correction, Image URL, Rights.

STEP 2 — ARTIST TRUTH TABLE (multi_llm.py)

```

ARTIST_TRUTH = {
  'Malevich': {
    'correct': 'Ukrainian-born (Kyiv, 1879); Polish ethnic heritage',
    'wrong': ['russian artist', 'russian avant-garde', 'russian modernism', 'russian painter'],
    'correct_kw': ['ukrainian', 'kyiv', 'kiev', 'polish'],
    'summary': (
      "Kazimir Malevich was born in Kyiv (then Kiev), capital of Ukraine, "
      "on 23 February 1879. He was of Polish ethnic heritage. The label "
      "'Russian artist' conflates the Soviet state with his Ukrainian origin."
    ),
  },
  'Exter': {
    'correct': 'Ukrainian-born (Kyiv); trained Kyiv School of Art',
    'wrong': ['russian artist', 'russian avant-garde', 'russian'],
    'correct_kw': ['ukrainian', 'kyiv', 'kiev'],
  },
  'Pagava': {
    'correct': 'Georgian-born (Tbilisi); emigre to Paris 1923',
    'wrong': ['russian', 'french artist', 'empire russe'],
    'correct_kw': ['georgian', 'tbilisi', 'georgia'],
  },
  'Kakabadze': {
    'correct': 'Georgian-born (Kutaisi/Tbilisi); Paris 1919-1927',
    'wrong': ['russian', 'soviet artist', 'russian avant-garde'],
    'correct_kw': ['georgian', 'kutaisi', 'tbilisi', 'georgia'],
  },
  'Parajanov': {
    'correct': 'Soviet-Armenian (Georgian-born); birth name Sarkis Parajanov',
    'wrong': ['soviet director', 'soviet filmmaker', 'soviet film'],
    'correct_kw': ['armenian', 'georgia', 'tbilisi', 'sarkis'],
  },
}

```

Each entry defines the ground truth against which AI responses are measured. The 'wrong' list contains keyword phrases that indicate misattribution. The 'correct_kw' list contains terms that must be present for a response to be considered accurate.

STEP 3 — QUERYING THREE AI SYSTEMS (multi_llm.py)

The same prompt is sent to all three models:

Prompt template:

```

PROMPT_TEMPLATE = (
  "I'm looking at '{title}' by {artist}. "
  "Can you tell me about this artist -- specifically their nationality, "
  "where they were born, where they trained, and how their cultural "
  "background shaped their work?"
)

```

Claude query:

```
def query_claude(artist, title, image_url=None):
    client = anthropic.Anthropic()
    prompt = PROMPT_TEMPLATE.format(title=title, artist=artist)
    msgs = []
    if image_url:
        msgs.append({"role": "user", "content": [
            {"type": "image",
             "source": {"type": "url", "url": image_url}},
            {"type": "text", "text": prompt},
        ]})
    else:
        msgs.append({"role": "user", "content": prompt})
    resp = client.messages.create(
        model="claude-opus-4-5",
        max_tokens=350,
        system="You are a knowledgeable art historian. "
              "Answer directly and specifically.",
        messages=msgs,
    )
    return resp.content[0].text
```

GPT-4o query:

```
def query_gpt(artist, title, image_url=None):
    client = openai.OpenAI(api_key=os.environ['OPENAI_API_KEY'])
    prompt = PROMPT_TEMPLATE.format(title=title, artist=artist)
    content = []
    if image_url:
        content.append({"type": "image_url",
                       "image_url": {"url": image_url}})
    content.append({"type": "text", "text": prompt})
    resp = client.chat.completions.create(
        model="gpt-4o",
        max_tokens=350,
        messages=[
            {"role": "system",
             "content": "You are a knowledgeable art historian."},
            {"role": "user", "content": content},
        ],
    )
    return resp.choices[0].message.content
```

LLAMA / Ollama query:

```
def query_ollama(artist, title):
    ollama_url = os.environ.get('OLLAMA_URL', 'http://localhost:11434')
    model      = os.environ.get('OLLAMA_MODEL', 'llama3')
    payload = json.dumps({
        "model": model,
        "prompt": "You are an art historian.\n\n"
                  + PROMPT_TEMPLATE.format(title=title, artist=artist),
        "stream": False,
    }).encode()
    req = urllib.request.Request(
        f"{ollama_url}/api/generate",
        data=payload,
        headers={"Content-Type": "application/json"},
        method="POST",
    )
    with urllib.request.urlopen(req, timeout=15) as resp:
        return json.loads(resp.read()).get('response', '')
```

STEP 4 — MISATTRIBUTION ASSESSMENT (multi_llm.py)

The `assess()` function compares the LLM response against the truth table using keyword matching:

```
def assess(artist, response_text):
    truth = ARTIST_TRUTH.get(artist, {})
    low   = response_text.lower()
    found_correct = [kw for kw in truth.get('correct_kw', [])
                     if kw in low]
    found_wrong   = [kw for kw in truth.get('wrong', [])
                     if kw in low]
    misattributed = bool(found_wrong) or not bool(found_correct)
    return {
        'misattributed':    misattributed,
        'correct_kw_found': found_correct,
        'wrong_kw_found':   found_wrong,
        'confidence':       0.91 if misattributed else 0.24,
    }
```

Logic: A response is flagged as misattributed if it contains any wrong-label keyword (e.g. "russian avant-garde") OR if it contains none of the correct keywords (e.g. "ukrainian", "kyiv"). Both conditions are necessary because a response can avoid wrong terms while still omitting the correct identity.

STEP 5 — COMPARE ALL THREE SYSTEMS (multi_llm.py)

```
def compare_all_llms(artist, image_url=None):
    records = [r for r in artworks_data['records']
                if r.get('_artist') == artist]
    title = records[0].get('Title', f'a work by {artist}') \
        if records else f'a work by {artist}'

    claude_resp = query_claude(artist, title, image_url)
    gpt_resp     = query_gpt(artist, title, image_url)
    llama_resp   = query_ollama(artist, title)

    return {
        'artist': artist,
        'systems': {
            'claude': {
                'name':      'Claude (Anthropic)',
                'response':  claude_resp,
                'assessment': assess(artist, claude_resp),
            },
            'gpt': {
                'name':      'GPT-4o (OpenAI)',
                'response':  gpt_resp,
                'assessment': assess(artist, gpt_resp),
            },
            'llama': {
                'name':      'LLAMA (Ollama)',
                'response':  llama_resp,
                'assessment': assess(artist, llama_resp),
            },
        },
        'truth': {
            'correct_label': ARTIST_TRUTH[artist]['correct'],
            'explanation':    ARTIST_TRUTH[artist]['summary'],
        },
    }
}
```

Returns a structured JSON object comparing all three AI responses with assessments. This is called by the `/api/compare` server endpoint and rendered in the visitor kiosk.

STEP 6 — HTTP API SERVER (server.py)

Python standard library HTTP server — no external frameworks. Routes:

Route	Method	Parameters	Returns
GET <code>/api/query</code>	GET	artist, image_url (optional)	Single Claude response + truth comparison
GET <code>/api/compare</code>	GET	artist, image_url (optional)	All three AI systems compared side by side
GET <code>/api/artworks</code>	GET	artist (optional filter)	Dataset records (max 50)
GET <code>/api/detect</code>	GET	url (image URL), artist	Misattribution detected true/false + confidence
GET <code>/api/images</code>	GET	artist (optional)	Image manifest, ready/total counts
GET <code>/api/stats</code>	GET	none	Dataset totals by artist

GET /api/sync	GET	none	Server timestamp for screen synchronisation
GET /	GET	none	Static files (HTML, JS, images)

Handler pattern:

```
class ExhibitionHandler(SimpleHTTPRequestHandler):
    def do_GET(self):
        parsed = urlparse(self.path)
        path = parsed.path
        params = parse_qs(parsed.query)

        if path == '/api/compare':
            artist = params.get('artist', ['Malevich'])[0]
            image_url = params.get('image_url', [None])[0]
            result = compare_all_llms(artist, image_url)
            self._json_response(result)

        elif path == '/api/stats':
            self._json_response({
                'total_records': len(artworks_data['records'])
                               + len(parajanov_data['records']),
                'by_artist': { ... }
            })

    def _json_response(self, data, status=200):
        body = json.dumps(data, ensure_ascii=False,
                          indent=2).encode('utf-8')
        self.send_response(status)
        self.send_header('Content-Type',
                        'application/json; charset=utf-8')
        self.send_header('Access-Control-Allow-Origin', '*')
        self.end_headers()
        self.wfile.write(body)
```

STEP 7 — IMAGE PIPELINE (image_pipeline.py)

Maps local image files to dataset records. Recognises filename prefixes:

Prefix	Artist	Example filename
MaL	Malevich	MaL01.jpg
std	Malevich (Stedelijk)	std03.jpg
EXT	Exter	EXT001.jpg
KAK	Kakabadze	KAK001.jpg
PAG	Pagava	PAG001.jpg
PAR	Parajanov	PAR001.jpg

```

ID_PATTERN = re.compile(
    r'(MAL|EXT|KAK|PAG|PAR)-\d{3}|'
    r'(MaL|std|EXT|KAK|PAG|PAR)\d+',
    re.IGNORECASE
)

def scan_local_dir(directory):
    for img_path in sorted(img_dir.rglob('*')):
        match = ID_PATTERN.search(img_path.stem)
        if match:
            prefix = re.match(r'[A-Za-z]+',
                               match.group()).group().upper()
            number = re.search(r'\d+', match.group())
            art_id = f"{prefix}-{number.group().zfill(3)}"
            artist = PREFIX_TO_ARTIST.get(prefix.lower())
            dest = IMAGES_DIR / f"{art_id}{img_path.suffix}"
            shutil.copy2(img_path, dest)
            index[art_id] = {
                'local_path': str(dest),
                '_artist': artist,
            }

```

Usage: `python3 server/image_pipeline.py --local-dir /path/to/images`

STEP 8 — PHASE STATE MACHINE (data.js)

The exhibition cycles through 8 phases automatically. Phase durations in milliseconds:

Phase	ID	Duration	Description
PRISTINE	0	3500ms	Particles coalesce into artwork form
LABEL_APPLY	1	3500ms	Archival stamp descends
CORRUPT	2	4500ms	Noise field tears particles apart
PROPAGATE	3	5000ms	Streams cross screens to archive nodes
MACHINE_GAZE	4	4500ms	Cold scanner reads corrupted data
INTERVENTION	5	3500ms	Warm force field reclaims particles
TRUTH	6	6000ms	Particles lock to correct positions
FADE	7	2000ms	Dissolves; next artist begins

```

const PHASES = {
  PRISTINE:    { id: 0, duration: 3500 },
  LABEL_APPLY: { id: 1, duration: 3500 },
  CORRUPT:     { id: 2, duration: 4500 },
  PROPAGATE:   { id: 3, duration: 5000 },
  MACHINE_GAZE: { id: 4, duration: 4500 },
  INTERVENTION: { id: 5, duration: 3500 },
  TRUTH:       { id: 6, duration: 6000 },
  FADE:        { id: 7, duration: 2000 },
};

```

STEP 9 — PARTICLE SYSTEM (sketch.js)

1,800 particles are seeded from artwork image pixels. Each particle has a target position (correct pixel location) and is subject to two competing forces:

```
function updateParticles(fw, fh, corruption, now) {
  const spring = lerp(0.055, 0.004, corruption);
  const damping = 0.86;
  const noiseAmp = lerp(0.0, 4.2, corruption);

  for (let i = 0; i < particles.length; i++) {
    const p = particles[i];

    // Spring toward correct target position
    p.vx += (p.tx - p.x) * spring;
    p.vy += (p.ty - p.y) * spring;

    // Noise turbulence -- misattribution force
    if (noiseAmp > 0.05) {
      const n1 = noise(p.x / 180, p.y / 180, noiseOffset);
      const n2 = noise(p.x / 180 + 50, p.y / 180, noiseOffset);
      p.vx += cos(n1 * TWO_PI * 3) * noiseAmp;
      p.vy += sin(n2 * TWO_PI * 3) * noiseAmp;
    }

    p.vx *= damping;
    p.vy *= damping;
    p.x += p.vx;
    p.y += p.vy;
  }
}
```

Corruption value (0.0 = truth, 1.0 = fully corrupted) drives both forces. When corruption = 1.0, spring strength drops to 0.004 and noise amplitude rises to 4.2, overpowering the spring.

Thread connections — nearby particles are connected by fine lines:


```

function drawThreads(corruption) {
  const hue = lerp(42, 195, corruption); // warm gold -> cold blue
  const maxD = lerp(52, 110, corruption); // wider mesh when corrupted
  const maxD2 = maxD * maxD;

  for (let i = 0; i < particles.length; i += 3) {
    const a = particles[i];
    let drawn = 0;
    for (let j = i+1; j < i+90 && drawn < 5; j++) {
      const b = particles[j];
      const dx = a.x - b.x;
      const dy = a.y - b.y;
      if (dx*dx + dy*dy < maxD2) {
        stroke(hue, sat, bri,
              map(dist, 0, maxD, baseAlpha, 0));
        line(a.x, a.y, b.x, b.y);
        drawn++;
      }
    }
  }
}

```

STEP 10 — MULTI-SCREEN SYNCHRONISATION (sync.js)

Screen 1 (leader) writes phase state to localStorage every 500ms. All other screens read and align:

```

function writeSync() {
  localStorage.setItem('misattribution_sync', JSON.stringify({
    phaseIndex: phaseIndex,
    artist: state.artist,
    phaseStart: state.phaseStart,
    ts: Date.now(),
  }));
}

function readSync() {
  const s = JSON.parse(localStorage.getItem('misattribution_sync'));
  if (Date.now() - s.ts > 3000) return; // stale -- run independently
  if (s.phaseIndex !== phaseIndex) {
    phaseIndex = s.phaseIndex;
    state.phase = phases[PHASE_ORDER[phaseIndex]];
    state.phaseStart = s.phaseStart;
  }
}

```

Fallback: if no sync signal within 3000ms, each screen runs the cycle independently.

STEP 11 — RUNNING THE SYSTEM

Environment setup:

```

export ANTHROPIC_API_KEY="sk-ant-..." # required for live AI
export OPENAI_API_KEY="sk-..." # optional for GPT-4o
python3 server/server.py

```

Link artwork images from local folder:

```
python3 server/image_pipeline.py --local-dir /path/to/images
```

Access points:

Interface	URL
Single screen preview	http://localhost:8000/index.html
Visitor kiosk (AI detection)	http://localhost:8000/kiosk.html
Curator controller	http://localhost:8000/exhibition.html
API test	http://localhost:8000/api/compare?artist=Malevich